TBSS Bernoulli Power Calculation

Catherine Mahoney

Power calculation for tree-based scan statistics with a Bernoulli probability model

This pipeline automates the calculation of power for tree-based scan statistics used in pharmacovigilance using the recently published, open-source implementation in the package `TBSS`. This version uses a Bernoulli probability model. To evaluate the power of the tree-based scan statistic, it implements a plasmode simulation procedure to introduce known elevation in relative risk, as well as a propensity matching procedure for the creation of matched cohorts, then calculate the proportion of trials in which the simulated node alerts. The pipeline uses the `targets` and `crew` packages to manage the parallel simulations and subsequent calculations.

Input data

The specification of the precise study design, including inclusion/exclusion criteria, baseline period, and observation period are out of the scope of this pipeline. It is assumed that the user has created the cohorts according to their selected design and formatted the table as below, with a column for *id*, *treatment_group*, each relevant covariate, and a single *outcome* column with one row for each patient-outcome combination.

This vignette uses a simulated data set of treatment group, covariates, and outcomes from a subset of ICD-10-CM diagnosis codes corresponding to diseases of the respiratory system (J00-J99). The subset of codes is used for demonstration purposes to reduce run time.

Sample input data

id	treatment_group	cov_1	cov_2	...	outcome
1	1	0	1	...	A047
1	1	0	1	...	J459
1	1	0	1	...	E119
2	0	1	0	...	M545
2	0	1	0	...	F321

Tree-structured variable

Tree-based scan statistics leverage the hierarchical nature of coding systems such as ICD-10-CM to test for excess risk as multiple resolutions. TBSS requires that the tree be formatted with a *parent* and *child* column, as shown below:

child	parent
J040	J04
J041	J04
J0410	J041
J0411	J041

The function `tree_builder()`, contained in the repository, accepts a vector of codes and generates an appropriately formatted tree-structured variable.

Pipeline steps

The example pipeline is detailed in the _targets.R file, and each target and corresponding function or input contains comments to guide users in incorporating their own data and study design choices. Briefly, the steps in each simulation are as follows:

1. `select_subsample()` Select treatment and comparator groups of the desired size; this example selects 1,000 treatment and 1,000 comparator patients
2. `plasmode()` Introduce known elevation in relative risk of an outcome using plasmode simulation. This example uses a relative risk of 2. The user can decide whether to select a specific outcome in which to simulate excess risk or to specify a target incidence rate for the outcome to enrich and allow the function to selcet the outcome randomly. In this instance, a target incidence of .01 is specified.
3. `ps_match()` Match treated and comparator patients based on a user-defined propensity score function as defined in `match_params.R`. In this example, the propensity score is computed as a logistic regression using sample baseline data corresponding to categories from the Elixhauser comorbidity index, but users should customise to their own study and may choose to evaluate differing degrees of control of potential confounding through choice of propensity score parameters. This example uses full matching with a caliper of 0.1, but the user may customise appropriately for their study design.
4. `tbss_mask()` Apply TBSS to determine if the simulated elevation in risk is detected while preventing detection of unrelated statistical alerts.

This process is repeated many times to determine the power, calculated as the proportion of simulations wherein the simulated excess risk is detected. The example uses only 10 simulations for demonstration purposes; suitable numbers will vary by application, but 1,000 may be a good reference point. The package `crew` manages parallelisation. In this example, only two workers are used, but the user can customise to their own computer system.

Modifying `match_params.R`

To specify the parameters for the stratification and estimation of expected outcome rates, the user must modify the parameters in the `match_params.R` script.

The sample arguments correspond to a binomial GLM with a logit link function, but the user can specify any valid arguments to `glm()` in the `match_args` list, along with the desired matching parameters to be passed to the `matchit()` function.

Contents of the `_targets.R` script

The script `_targets.R` directs the flow of work in the pipeline. Once the matching parameters in `match_params.R` have been specified, lines with a comment can be customised to required inputs and file locations for input and tree data.

```
library(targets)
library(crew)

tar_source() #configure metadata with matching parameters
tar_option_set(packages = c("readr", "dplyr", "MatchIt", "TBSS", "optmatch"),
               controller = crew_controller_local(workers = 2) #specify workers
)

list(
  tar_target(file,
             "data/test_data.csv",    #location of your outcome and covariate data
             format = "file"
  ),
  tar_target(data,
             get_data(file)
  ),
  tar_target(iteration_id,
             1:10                #specify number of replicates
  ),
  tar_target(subsample,
             select_n(data, 1000, 1000),    #specify sample sizes
```

```
            pattern = map(iteration_id)
  ),
  tar_target(sim_data,
            plasmode(subsample,
                     target_outcome = NULL, #specify either outcome or incidence
                     target_inc = .01,
                     rr=2),                 #specify relative risk
            pattern = map(subsample)
  ),
  tar_target(ps_data,
            ps_match(sim_data,
                     match_args),
            pattern = map(sim_data)
  ),
  tar_target(tree_file,
            'data/test_tree.csv',                   #location of the tree file
            format = "file"
  ),
  tar_target(tree,
            get_data(tree_file)
  ),
  tar_target(tbss,
            tbss_mask(ps_data,
                      model = "bernoulli",
                      tree),
            pattern = map(ps_data)
  ),
  tar_target(final_vector,
            unlist(tbss)
  ),
  tar_target(final_power,
            print(mean(final_vector))
  )

)
```

Running the pipeline

Run `tar_make()` to launch the pipeline. The output will enumerate each target and show which branches were already up-to-date. Finally, `tar_read(final_power)` will display the result of the complete power calculation. Here, TBSS had 20% power to detect an outcome

with an incidence of .01 and a relative risk of 2 when there were 1,000 treated and 1,000 comparator patients.

```
tar_make()
```

```
  start target iteration_id
  start target tree_file
  built target iteration_id [1.076 seconds]
  start target file
  built target tree_file [0.001 seconds]
  start target tree
  built target file [0 seconds]
  start target data
  built target tree [0.332 seconds]
  built target data [2.266 seconds]
  start branch subsample_62e81964
  start branch subsample_b4bfd040
  start branch subsample_38a79088
  built branch subsample_62e81964 [0.223 seconds]
  start branch sim_data_c9704089
  built branch subsample_b4bfd040 [0.249 seconds]
  start branch sim_data_0cea77f1
  built branch subsample_38a79088 [0.33 seconds]
  start branch sim_data_85fd2479
  built branch sim_data_c9704089 [0.265 seconds]
  built branch sim_data_0cea77f1 [0.412 seconds]
  start branch ps_data_60f56dbc
  built branch sim_data_85fd2479 [0.483 seconds]
  start branch ps_data_761d80a0
  built branch ps_data_60f56dbc [1.595 seconds]
  start branch tbss_10f16beb
  built branch ps_data_761d80a0 [1.639 seconds]
  start branch tbss_306f3d59
  built branch tbss_306f3d59 [9.59 seconds]
  start branch ps_data_25e8574f
  built branch tbss_10f16beb [10.614 seconds]
  start branch subsample_9cda9fa1
  built branch subsample_9cda9fa1 [0.061 seconds]
  start branch sim_data_dc21cee1
  built branch ps_data_25e8574f [1.125 seconds]
  start branch tbss_410784a2
  built branch sim_data_dc21cee1 [0.215 seconds]
```

```
start branch ps_data_c2298b02
built branch ps_data_c2298b02 [1.558 seconds]
start branch tbss_40a1c294
start branch subsample_d8835e12
built branch tbss_410784a2 [10.098 seconds]
built branch subsample_d8835e12 [0.1 seconds]
start branch sim_data_85411e4b
built branch sim_data_85411e4b [0.281 seconds]
start branch ps_data_c1d5291e
built branch tbss_40a1c294 [9.284 seconds]
start branch subsample_fb5d7868
built branch subsample_fb5d7868 [0.068 seconds]
start branch sim_data_7e93bdc4
built branch sim_data_7e93bdc4 [0.38 seconds]
start branch ps_data_dda3b972
built branch ps_data_c1d5291e [1.364 seconds]
start branch tbss_bb2d41e4
built branch ps_data_dda3b972 [1.095 seconds]
start branch tbss_bd620b9b
built branch tbss_bb2d41e4 [10.716 seconds]
start branch subsample_886a580b
built branch tbss_bd620b9b [10.062 seconds]
start branch subsample_1b97e11f
built branch subsample_886a580b [0.076 seconds]
start branch sim_data_e0dd2595
built branch subsample_1b97e11f [0.474 seconds]
start branch sim_data_788d8f30
built branch sim_data_e0dd2595 [0.406 seconds]
start branch ps_data_c8fcfa9e
built branch sim_data_788d8f30 [0.221 seconds]
start branch ps_data_30a6f5b3
built branch ps_data_c8fcfa9e [1.162 seconds]
start branch tbss_cb746b36
built branch ps_data_30a6f5b3 [1.148 seconds]
start branch tbss_2b6cf182
built branch tbss_cb746b36 [9.534 seconds]
start branch subsample_2db33782
built branch subsample_2db33782 [0.063 seconds]
start branch sim_data_c5f5adf9
built branch tbss_2b6cf182 [9.965 seconds]
start branch subsample_e2199db5
built branch subsample_e2199db5 [0.064 seconds]
built pattern subsample
```

```
  built branch sim_data_c5f5adf9 [0.247 seconds]
  start branch ps_data_e66d3b3b
  start branch sim_data_20ee2357
  built branch sim_data_20ee2357 [0.365 seconds]
  built pattern sim_data
  start branch ps_data_8115835d
  built branch ps_data_e66d3b3b [1.332 seconds]
  start branch tbss_81b8b7a8
  built branch ps_data_8115835d [1.882 seconds]
  built pattern ps_data
  start branch tbss_d06d9655
  built branch tbss_81b8b7a8 [9.486 seconds]
  built branch tbss_d06d9655 [10.236 seconds]
  built pattern tbss
  start target final_vector
  built target final_vector [0.001 seconds]
  start target final_power
  built target final_power [0 seconds]
  end pipeline [1.295 minutes]
Warning message:
10 targets produced warnings. Run targets::tar_meta(fields = warnings, complete_only = TRUE)

[1] "Final power: 0.2"
```